



RevisionFlow

Planificateur de révisions intelligent

NOUGBOLO Godwin Elie

Encadrante : Pr. RABHI Loubna

ENSA Khouribga – IID1

2025 – 2026

Sommaire

- 1 Le problème
- 2 Présentation de l'application
- 3 Cœur algorithmique
- 4 Architecture technique
- 5 Conclusion

Le problème réel des étudiants ingénieurs

La situation typique :

- 5 à 8 examens en quelques semaines
- Niveaux de difficulté très différents
- Imprévus fréquents (date déplacée, cours annulé)

Les outils classiques échouent car :

- × Agenda papier : rigide, recalcul manuel
- × Tableur : aucune priorisation automatique
- × Aucun outil ne détecte une charge **irréaliste**

Trop de matières →
mauvaise priorisation
→ retard non détecté

Résultat :

Panique de dernière minute

Pourquoi pas Google Calendar ou Notion ?

Critère	Agenda	GCalendar	Notion	RevisionFlow
Génération automatique	×	×	×	✓
Recalcul après imprévu	×	×	×	✓
Détection de surcharge	×	×	×	✓
Priorisation intelligente	×	×	Partielle	✓
Zéro inscription / hors ligne	✓	×	×	✓

Notre réponse

Un planificateur automatique, transparent et réactif — conçu spécifiquement pour les contraintes des examens.

Trois principes non négociables

Zéro friction

Aucune inscription.
Données
100 % locales.
Fonctionne
hors ligne.

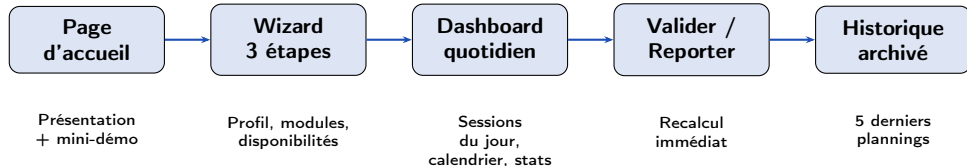
Transparence

Planning irréaliste ?
L'app le dit
clairement.
Jamais de mensonge
doux.

Réactivité

Chaque action déclenche
un recalcul immédiat
du planning futur.

Le parcours de l'étudiant en un coup d'œil



Ce que fait l'algorithme à chaque étape

Wizard → génération initiale seule | Action → recalcul du futur uniquement | Historique → lecture

Wizard — Étape 1 : Profil

- **Type** : Fixe ou Flexible
 - *Fixe* : dates imposées (concours)
 - *Flexible* : dates modifiables librement, recalcul immédiat si déplacées
- **Pays** : récupère automatiquement les jours fériés
- **Date de début** du planning

RevisionFlow

Étape 1 sur 3

1 PROFIL 2 MODULES 3 DISPONIBILITÉS

Ton profil étudiant
Configure ta situation académique en 3 étapes.

TYPE D'ORGANISATION DES EXAMENS

Dates flexibles
Les dates d'examen sont flexibles par les profs et peuvent changer.

Emploi du temps fixe
Les dates sont imposées par l'administration et connues à l'avance.

ZONE GÉOGRAPHIQUE

Ton pays (pour les jours fériés)

M. Maroc

Jours fériés — API Nager.Date

GET /api/v3/PublicHolidays/2025/MA

→ traités comme des week-ends (haute capacité)

→ récupérés pour l'année en cours **et** la suivante

Wizard — Étapes 2 & 3

Étape 2 : Modules

- Nom, date d'examen
- Difficulté (1 à 5 étoiles)
- Nombre de chapitres
- Indicateur de priorité calculé **en temps réel**

The progress bar shows three stages: 'Personne' (100% complete), 'Module' (50% complete), and 'Disponibilité' (0% complete). Below it, a card titled 'Tes modules' lists the 'Dev web' module. The card displays the exam date as '04/06/2026', the number of chapters as '4', and the difficulty level as '5 stars'. A red banner indicates 'Urgent ! Prochaine heure'. At the bottom of the card is a button to 'Ajouter un module'.

Étape 3 : Disponibilités

- Sessions/soir en semaine (défaut : 2h)
- Sessions le week-end (défaut : 6h)
- Durée par chapitre (défaut : 1h)
- Blocage manuel de jours de repos

RevisionSaver

Écran 1 sur 3

Tes disponibilités

Configure ton rythme de travail et bloque tes jours d'indisponibilité.

VOLUME Horaire Quotidien

Sonée
Evening

- 1.50h +

Weekend
Samedi & Dimanche

- 6.50h +

Session
Full Day 1

- 1h +

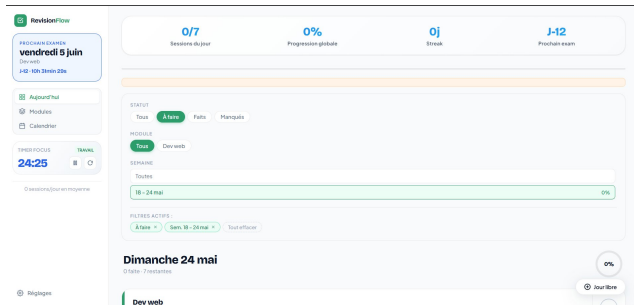
JOURS D'INDISPONIBILITÉ

Mar 2026

12/1	10/1	10/2	20/1	10/3	20/4	20/5
				1	2	3
4	5	6	7	8	9	10

Dashboard — Vue « Aujourd'hui »

- **Barre de stats** : sessions faites/total, streak, jours avant le prochain examen
- **Anneau de progression** du jour
- **Sessions triées** par priorité dynamique
- **Aperçu** des 5 prochains jours
- **Minuteur Pomodoro** en sidebar
 - 25 min travail / 5 min pause
 - Son via Web Audio API à la fin
 - Propose de valider la session correspondante



Actions sur une session

Bouton	Action	Effet immédiat sur le planning
✓	Valider	Compteur du module +1, recalcul du futur
🔄	Reporter	Session remplacée automatiquement selon la priorité
	Anticiper	Avance une session du lendemain si capacité disponible
+	Jour libre	Journée transformée en haute capacité pour rattraper un retard

Principe clé

Chaque action modifie **uniquement le futur**. Le passé est toujours conservé intact pour la

Filtres combinables & Calendrier

Filtres simultanés sur 3 axes

- **Statut** : Tous / À faire / Faits / Manqués
- **Module** : Tous / un module spécifique
- **Semaine** : Toutes / une semaine précise

Les filtres actifs sont persistés entre les sessions.

Calendrier mensuel — code couleur

- Journée complète
- En cours (futur)
- Sessions manquées
- Repos
- Date d'examen

Persistence des données

localStorage JSON · Export/Import .json · Validation de structure à l'import · 5 derniers plannings archivés en lecture seule

L'exemple que l'on va suivre jusqu'au bout

Situation de départ — lundi matin

Un étudiant a **3 modules** à préparer. Sa capacité : **2 sessions/soir** en semaine, **4 sessions** le week-end.

Module	Difficulté	Chapitres	Examen dans	Charge brute
Analyse	4	6	5 jours	$4 \times 6 = 24$
Algorithmes	3	5	12 jours	$3 \times 5 = 15$
Physique	2	3	20 jours	$2 \times 3 = 6$
Total	14 sessions			

Pourquoi un algorithme déterministe, pas du ML ?

Algorithme glouton (notre choix)

- ✓ Règles explicites, résultat prévisible
- ✓ Aucune donnée d'entraînement nécessaire
- ✓ Exécution instantanée, hors ligne
- ✓ L'étudiant comprend *pourquoi* telle session est placée là
- ✓ Même entrée → même planning, toujours

Machine Learning (écarté)

- ✗ Besoin de milliers d'exemples de plannings parfaits
- ✗ Boîte noire : pas de transparence
- ✗ Risque de plannings incohérents
- ✗ Complexité injustifiée pour un problème d'ordonnancement sous contraintes explicites

Conclusion

Un problème d'optimisation sous contraintes **explicites** se résout très bien sans IA. La transparence et la maîtrise de l'étudiant priment sur la sophistication apparente.

Étape 1 — Score de priorité : objectiver l'urgence

Pourquoi une formule ? Sans calcul, l'étudiant choisit instinctivement ce qu'il préfère, pas ce qui est urgent. La formule impose une priorisation **objective**.

$$P_i = \underbrace{(\text{difficulté}_i \times \text{chapitres}_i)}_{\substack{\text{charge brute} \\ \text{effort total requis pour ce module}}} \times \underbrace{\left(1 + \frac{1}{\ln(\Delta t_i + 2)}\right)}_{\substack{\text{multiplicateur d'urgence} \\ \text{pression temporelle, amortie par } \ln}}$$

Pourquoi le logarithme ?

- Sans lui : un examen dans 1 jour aurait un score **infiniment** plus grand que dans 2 jours
- Le **ln** **amortit** l'explosion tout en maintenant une vraie pression temporelle
- Le planning reste **équilibré** même en urgence extrême

Sur notre exemple (lundi, Δt en jours)

Module	Δt	Mult.	Score P
Analyse	5	1.51	36.3
Algorithmes	12	1.37	20.6
Physique	20	1.32	7.9

Étape 2 — Entrelacement : varier les matières dans la journée

Problème sans entrelacement : avec Analyse à 36.3, l'algorithme naïf placerait Analyse sur toutes les sessions du jour. C'est contre-productif pour la rétention mémorielle.

Solution : chaque session supplémentaire du même module dans la même journée est pénalisée par 2^k :

$$P_i^{\text{jour}} = \frac{P_i}{2^k} \quad (k = \text{nombre de sessions déjà attribuées à } i \text{ ce jour})$$

Simulation — Lundi (capacité = 2)

Slot	P_A	P_{AI}	P_P	Élu	k
1	36.3	20.6	7.9	A	$0 \rightarrow 1$
2	18.2	20.6	7.9	AI	$0 \rightarrow 1$

Lundi : **Analyse, Algorithmes**

Alternance naturelle malgré l'écart de score.

Pourquoi c'est important

- Évite 2h d'affilée de la même matière
- Améliore la rétention mémorielle (principe d'apprentissage entrelacé, Kornell & Bjork, 2008)
- Fonctionne **automatiquement**, sans action de l'étudiant

Étape 3 — Algorithme glouton : remplir les jours un par un

Boucle principale

- ① Générer tous les jours jusqu'au dernier examen
- ② Pour chaque jour (ordre chronologique) :
 - Tant que capacité disponible :
 - ① Recalculer P_i^{jour} pour chaque module
 - ② Trier : score ↓, chapitres ↓, nom ↑ (*déterminisme*)
 - ③ Placer la session la plus prioritaire
- ③ Sessions restantes → backlog

Capacité selon le type de jour

Soir semaine	heuresSoir
Week-end	heuresWeekend
Jour férié	= Week-end (via Nager.Date)

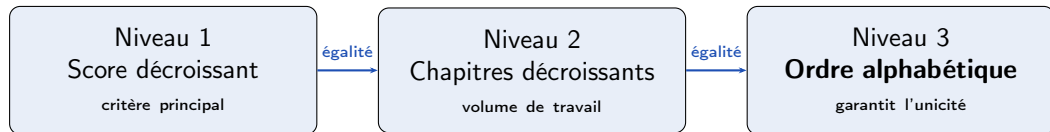
Résultat sur notre exemple

Jour	Cap.	Sessions
Lundi	2	A, Al
Mardi	2	A, Al
Mercredi	2	A, Al
Jeudi	2	A, Al
Vendredi	2	A, Al
Samedi	4	Al, P, Al, P
Dimanche	4	P, Al, P, —

Analyse (6 chap.) épuisée en semaine.

Le tri déterministe — un détail qui compte

Problème : deux modules peuvent avoir exactement le même score. Sans règle de départage, le planning change à chaque recalcul.



Pourquoi l'ordre alphabétique ?

C'est le seul critère **toujours défini, stable** et **indépendant** des données métier. Il garantit qu'une même entrée produit *toujours* le même planning.

Conséquence

L'algorithme est **déterministe** : reproductible, testable, et prévisible. L'étudiant peut anticiper l'ordre des sessions.

Backlog — la transparence avant tout

Variante de notre exemple

Supposons qu'Analyse ait **10 chapitres** au lieu de 6, avec l'examen dans **3 jours**.

Capacité : $3 \times 2 = 6$ sessions max

Sessions nécessaires : **10**

Backlog : 4 sessions impossibles à placer

Réaction de l'application

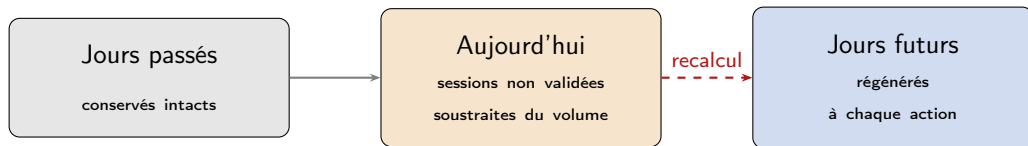
- Toast rouge affiché immédiatement :
« 4 sessions ne pourront pas être planifiées »
- L'étudiant choisit consciemment :
 - Augmenter sa capacité journalière
 - Réduire le nombre de chapitres
 - Déclarer un jour libre (+)

Philosophie

L'application ne cache jamais une surcharge.
Elle force une **décision consciente**.

Préserver le passé, recalculer uniquement le futur

Contrainte fondamentale : à chaque action, tout le planning est recalculé, mais les jours passés ne doivent **jamais** être modifiés.



Cohérence des stats

Streak, taux de complétion et vitesse restent fiables.

Pas de double-comptage

Les sessions du jour non validées sont soustraites avant de régénérer le futur.

Motivation préservée

L'étudiant n'est jamais « puni » par son historique.

Vue d'ensemble — technologies et choix

Stack technique

HTML5 / CSS3	Structure, design system
JavaScript ES6+	Logique métier, algorithmes
localStorage	Persistance locale
Nager.Date API	Jours fériés par pays
Web Audio API	Sons du Pomodoro
canvas-confetti	Animation fin de journée

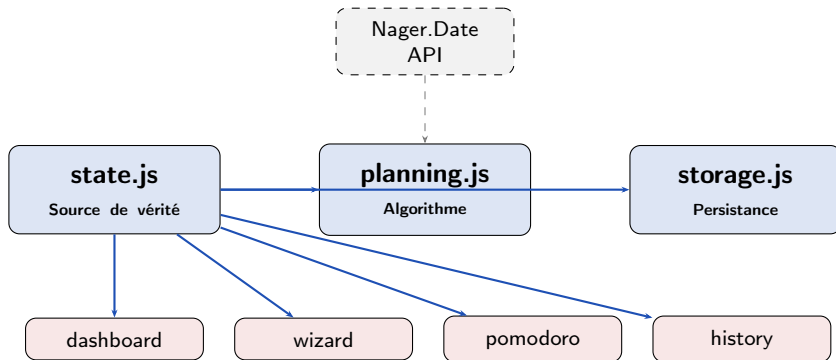
Pourquoi ces choix ?

- **localStorage** : données 100 % locales, aucune dépendance serveur, disponible hors ligne
- **Nager.Date** : API publique gratuite, couvre 100+ pays, réponse en JSON simple
- **Web Audio API** : native au navigateur, aucune librairie sonore à charger
- **Vanilla JS** : zéro dépendance externe, bundle final = 0 KB de framework

Aucun framework

Pas de React, Vue, ou Angular. Architecture modulaire en JS pur, ce qui facilite la portabilité future vers React Native ou Flutter.

Architecture modulaire — schéma des dépendances



Pattern Store — source de vérité unique

`State.get()` lecture seule · `State.update()` mise à jour + sauvegarde automatique ·
`State.subscribe(fn)` réactivité des composants · `State.planifier()` déclenche l'algorithme

Le pattern Store — pourquoi c'est important

Problème classique : dans une app avec plusieurs composants (dashboard, wizard, pomodoro...), comment éviter que chacun gère son propre état et que tout diverge ?

Sans pattern Store

- ✗ Chaque composant stocke ses propres données
- ✗ Synchronisation manuelle et fragile
- ✗ Un bug dans un composant corrompt les autres
- ✗ Impossible de tester isolément

Avec notre pattern Store

- ✓ Un seul objet State, une seule source de vérité
- ✓ Sauvegarde automatique à chaque `update()`
- ✓ Les composants s'abonnent et se mettent à jour automatiquement
- ✓ Inspiré de Redux, sans ses dépendances

Résultat concret

Le Pomodoro, le dashboard et l'historique affichent **toujours** les mêmes données, sans jamais se désynchroniser.

Défis techniques résolus

Migration des données sauvegardées

Si la structure de l'état change entre deux versions, les anciennes sauvegardes sont incomplètes.

Solution : `deepMerge(sauvegardé, initial)` — les nouvelles propriétés reçoivent leur valeur par défaut.

Synchronisation Pomodoro ↔ planning

Le minuteur tourne en continu. À sa fin, il doit valider la *bonne* session sans couplage direct.

Solution : à la fin d'un Pomodoro, interroger `State.get()` pour trouver la première session non validée du jour.

Limites connues et assumées

- `localStorage` limité à ~5 MB
- Pas de synchronisation multi-appareils
- Pas de notifications push hors onglet

Perspectives directes

- PWA : installation mobile + hors ligne
- Tests unitaires avec Vitest
- Internationalisation (anglais, arabe)
- Import d'emploi du temps iCal
- `planning.js` portable vers React Native / Flutter (aucune dépendance au DOM)

Ce que ce projet m'a appris

Algorithmique

- Concevoir un algorithme glouton déterministe
- Gérer un tri multi-critères pour garantir l'unicité
- Équilibrer urgence et volume via une formule mathématique

Architecture logicielle

- Pattern Store sans framework
- Séparation nette logique métier / DOM
- Gestion d'état global reproductible

Expérience utilisateur

- Concevoir pour réduire la friction
- Anticiper les cas limites (backlog, migration)
- Rendre une surcharge lisible plutôt que de la cacher

Web et APIs

- Intégration d'API externe (Nager.Date)
- Web Audio API native
- Design system CSS avec mode sombre

RevisionFlow en résumé

Pour l'étudiant

successgreen✓ Planning généré
automatiquement en 3
minutessuccessgreen✓ Priorisation basée
sur difficulté *et*
urgencesuccessgreen✓ Adaptabilité totale
aux imprévussuccessgreen✓ Transparence
sur la charge
réellesuccessgreen✓ Motivation via streak,
animations, Pomodoro

Techniquement

accentblue● 100 % JavaScript vanilla,
aucun frameworkaccentblue● Architecture
modulaire et
extensibleaccentblue● Algorithme glouton
+ entrelacement + tri
déterministeaccentblue● Données 100 %
locales, fonctionne hors
ligneaccentblue● Prêt pour PWA et mobile
natif

RevisionFlow

Votre assistant révisions — intelligent, transparent, et entièrement local.

Merci pour votre attention !

Des questions ?

GitHub

